

Business Intelligence Agent - Decision Log

The agent that I used is LangGraph ReAct agent, hosted by Groq (qwen/qwen3.6-27b). This is bound to monday.com's hosted MCP server. It is only a read-only tool and answers the founder-level questions across the two monday boards, Work orders and Deals. It generates and runs a read-only GraphQL with the help of the `execute_code` tool.

Key Assumptions:

1. When manually analyzing the database, item names were masked placeholders and are not unique or usable.
2. No per-work-order identifier exists on the Deals board. The only reliable link between boards is customer identity. This is shown by the Work Orders' Customer Name Code and the Deals' Client Code.
3. Working with MCP: MCP's tool-calling model lets the LLM decide which board/tool to use per question, rather than hand-writing a fixed function per query type.
4. LLM choice: qwen/qwen3.6-27b via Groq was chosen.
5. Because the integration is strictly read-only, the agent's role with messy data is to flag, not to silently clean or rewrite records on the board.

Trade-offs:

- **Tool allowlist enforced in code, not just via token scope.** Even with the Monday API token being read-only, the agent's tool list was limited to `get_board_info` and `execute_code`. This extra measure ensures it can't call any write, create, or delete tools. Three tools (`get_updates`, `get_board_activity`, `list_users_and_teams`) were removed when their schema size began to impact Groq's token-per-minute limit.
- **Model selection under real constraints.** qwen/qwen3.6-27b was selected for reliable tool-calling. Tried using llama-3.3-70b-versatile as it has a higher free-tier rate limit (12,000 vs 8,000 tokens/minute), but Groq decommissioned that model mid-project. So has to switch back to qwen3.6-27b, which reintroduced the tight token budget as an ongoing constraint.
- **Token budget.** With two boards' schemas and Monday's MCP tool descriptions, a single request could exceed the 8,000 token/minute ceiling before any real data was fetched. Cross-board join questions were particularly problematic, pulling from both boards. Solutions included: trimming the tool allowlist, tightening the system prompt's wording, capping conversation history to the last 12 messages, and most importantly, directing the agent's Python to aggregate before printing. Raw item dumps, not the system prompt, caused cross-board query failures.
- **Streamlit for the hosted UI.** Streamlit was chosen for the quickest path to a testable, hosted link. Each question reconnects to the MCP server fresh in a single `asyncio.run()` call, avoiding connection caching across reruns. Streamlit reruns the entire script for each interaction, and the MCP client holds resources tied to the event loop that created it. This avoids async bugs but adds a slight latency cost per question.

Interpretation: “Help Prepare Data for Leadership Updates”

The assumption is that this is a structured summary for leadership updates. It covers pipeline health, operational status, cross-board risks, and data caveats. A founder can use it directly in a leadership deck or email with little editing.

What I'd Do Differently With More Time

- Upgrade from Groq's free tier (Developer tier: no minimum spend, ~10x rate limits) to eliminate the token-budget issue.
- Implement real normalization for inconsistent fields. For example, convert various Collection Date formats to a standard date, and extract the quantity and unit from the free-text PO quantity field. This would allow the agent to compute directly rather than decline or hedge.
- Add retry/backoff and clearer error handling for Monday MCP connection failures and timeouts, beyond the current rate-limit handling.
- Test systematically across all question types mentioned in the brief—revenue, pipeline health, sector performance, and operational metrics—rather than focusing mainly on connection and schema verification.